

Computer Networks Mini Project: Distributed Log Aggregation Server

Code Explanation (server.c)

This document provides a block-by-block explanation of the `server.c` code, designed to be simple, clear, and easy to understand for presentations and exams.

Block 1: Header Files and Definitions

Plain Text

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <arpa/inet.h>
#include <unistd.h>
#include <time.h>

#define PORT 5000
#define BUFFER_SIZE 1024
```

Explanation:

- **Header Files (`#include ...`):** These bring in standard libraries needed for the program.
 - `<stdio.h>` & `<stdlib.h>` : For standard input/output (like `printf`) and general functions (like `exit`).
 - `<string.h>` : For string manipulation.
 - `<arpa/inet.h>` : Crucial for network programming (provides structures and functions for IP addresses and ports).
 - `<unistd.h>` : Provides access to POSIX operating system API (like `close` for sockets).
 - `<time.h>` : Used for getting the current time to timestamp the logs.
- **Macros (`#define ...`):**
 - `PORT 5000` : The server will listen on port 5000. Clients must send data to this specific port.
 - `BUFFER_SIZE 1024` : The maximum size of a single log message the server can receive at once (1024 bytes).

Block 2: Main Function and Variable Declarations

Plain Text

```
int main() {
    int sockfd;
    struct sockaddr_in server_addr, client_addr;
    char buffer[BUFFER_SIZE];
    socklen_t addr_len = sizeof(client_addr);

    long total_bytes = 0;
    int log_count = 0;
    time_t start_time = time(NULL);
```

Explanation:

- **sockfd** : A file descriptor (an integer ID) that will represent our network socket.
- **server_addr & client_addr** : Structures that hold IP address and port information for the server and the connecting client, respectively.
- **buffer** : An array to temporarily store the incoming log messages.
- **addr_len** : Stores the size of the client address structure, required by the receiving function.
- **total_bytes & log_count** : Variables to keep track of statistics (total data received and number of logs).
- **start_time** : Records the exact time the server started, used later to calculate throughput (data speed).

Block 3: Socket Creation

Plain Text

```
// Create UDP socket
sockfd = socket(AF_INET, SOCK_DGRAM, 0);

if (sockfd < 0) {
    perror("Socket creation failed");
    exit(EXIT_FAILURE);
}
```

Explanation:

- **socket(...)** : This creates the endpoint for communication.
 - **AF_INET** : Specifies IPv4 addressing.
 - **SOCK_DGRAM** : Specifies that this is a **UDP** socket (Datagram-based, connectionless).

- `0` : Uses the default protocol for UDP.
- **Error Handling:** If `socket()` returns a negative number, it means creation failed. The program prints an error and exits.

Block 4: Configuring Server Address

Plain Text

```
// Configure server address
server_addr.sin_family = AF_INET;
server_addr.sin_port = htons(PORT);
server_addr.sin_addr.s_addr = INADDR_ANY;
```

Explanation:

- `sin_family = AF_INET` : Sets the address family to IPv4.
- `sin_port = htons(PORT)` : Sets the port number (5000). `htons()` (Host TO Network Short) ensures the port number is in the correct byte order for network transmission.
- `sin_addr.s_addr = INADDR_ANY` : Tells the server to listen on **all available network interfaces** (IP addresses) of the machine.

Block 5: Binding the Socket

Plain Text

```
// Bind socket
if (bind(sockfd, (struct sockaddr *)&server_addr, sizeof(server_addr)) < 0)
    perror("Bind failed");
    exit(EXIT_FAILURE);
}
printf("Distributed Log Aggregation Server Started\n");
```

Explanation:

- `bind(...)` : This attaches the socket (`sockfd`) to the specific IP address and port we configured in `server_addr` . It essentially reserves port 5000 for this program.
- If binding fails (e.g., port 5000 is already in use by another program), it prints an error and exits.

Block 6: The Infinite Loop and Receiving Data

Plain Text

```

while (1) {
    int bytes = recvfrom(sockfd, buffer, BUFFER_SIZE - 1, 0,
                        (struct sockaddr *)&client_addr, &addr_len);

    if (bytes < 0) {
        perror("Receive failed");
        continue;
    }
    buffer[bytes] = '\0';
}

```

Explanation:

- **while (1)** : An infinite loop. A server runs continuously, always waiting for new data.
- **recvfrom(...)** : This function blocks (waits) until a UDP packet arrives.
 - It reads up to `BUFFER_SIZE - 1` bytes into the `buffer`.
 - It also fills `client_addr` with the sender's IP and port.
 - It returns the number of `bytes` actually received.
- **buffer[bytes] = '\0'** : Adds a null terminator to the end of the received data so it can be safely treated and printed as a standard C string.

Block 7: Processing and Displaying Logs

Plain Text

```

// Update statistics
total_bytes += bytes;
log_count++;

// Timestamp
time_t now = time(NULL);
struct tm *tm_info = localtime(&now);
char time_str[30];
strftime(time_str, sizeof(time_str), "%Y-%m-%d %H:%M:%S", tm_info);

printf("[%s] %s\n", time_str, buffer);

```

Explanation:

- **Statistics:** Updates the total bytes received and increments the log counter.
- **Timestamping:**
 - Gets the current time (`now`).
 - Converts it to local time format (`localtime`).

- Formats it into a readable string like "YYYY-MM-DD HH:MM:SS" (`strftime`).
- **Printing:** Displays the formatted timestamp followed by the actual log message received from the client.

Block 8: Calculating Throughput

Plain Text

```
// Calculate throughput
double elapsed_time = difftime(now, start_time);

if (elapsed_time > 0) {
    double throughput = total_bytes / elapsed_time;
    printf("Logs received: %d | Total Data: %ld bytes | Throughput: %.2f\n",
          log_count, total_bytes, throughput);
}
```

Explanation:

- `difftime(...)` : Calculates how many seconds have passed since the server started.
- **Throughput:** Calculates the data rate (Total Bytes / Elapsed Time) in bytes per second.
- **Printing Stats:** Displays the total logs, total data, and current throughput.

Block 9: Cleanup (Unreachable in this code)

Plain Text

```
close(sockfd);
return 0;
}
```

Explanation:

- `close(sockfd)` : Closes the socket to free up system resources.
- *Note:* Because of the `while(1)` infinite loop, the program never actually reaches these lines unless it is forcefully terminated (e.g., pressing Ctrl+C).

Important Viva Questions & Answers

1. What is a Server?

Answer: A server is a computer program or device that provides a service, data, or resources to other programs or devices, known as "clients," over a network. In this project, our server listens for incoming log messages from clients and aggregates (collects and displays) them.

2. What is UDP (User Datagram Protocol)?

Answer: UDP is a connectionless communication protocol used in networking.

- **Connectionless:** It does not establish a dedicated connection before sending data (unlike TCP).
- **Fast but Unreliable:** It just sends packets (datagrams) without checking if they arrived successfully. There is no acknowledgment, no retry mechanism, and packets might arrive out of order.
- **Why use it here?** For log aggregation, speed is often more important than perfect reliability. Losing a single log message occasionally is usually acceptable compared to the overhead of establishing a TCP connection for every small log entry.

3. What is the difference between TCP and UDP?

Answer:

- **TCP (Transmission Control Protocol):** Connection-oriented, reliable, guarantees delivery and order, but is slower due to overhead (e.g., 3-way handshake).
- **UDP (User Datagram Protocol):** Connectionless, unreliable, does not guarantee delivery or order, but is very fast and lightweight.

4. What is a Socket?

Answer: A socket is an endpoint for sending or receiving data across a computer network. It is a combination of an IP address and a Port number. It acts as a "door" through which an application can send and receive messages.

5. What is the role of `bind()` in the server code?

Answer: The `bind()` function associates the created socket with a specific local IP address and Port number (in our case, port 5000). This tells the operating system: "Route any incoming network traffic destined for port 5000 to this specific program."

6. Why do we use `htons()` for the port number?

Answer: `htons` stands for "Host TO Network Short". Different computers store numbers in memory differently (Little Endian vs. Big Endian). Networks use a standard format called

"Network Byte Order" (Big Endian). `htons()` converts our port number into this standard network format so any computer can understand it.

7. What does `INADDR_ANY` mean?

Answer: `INADDR_ANY` tells the server to listen for incoming connections on **all** available network interfaces (IP addresses) of the machine it is running on. This is useful if the server has multiple IP addresses (e.g., localhost and a public IP).

8. What is the difference between `recv()` and `recvfrom()` ?

Answer:

- `recv()` is typically used with connected sockets (like TCP) where the sender is already known.
- `recvfrom()` is used with connectionless sockets (like UDP). It not only receives the data but also captures the IP address and port of the sender, which is necessary since there is no established connection.

9. Why do we add `\0` (null terminator) to the buffer after receiving data?

Answer: In C, strings must end with a null character (`\0`) so functions like `printf` know where the string ends. The `recvfrom()` function just receives raw bytes and does not automatically add this terminator. If we don't add it manually, `printf` might print garbage values from memory past the end of our received data.

10. What happens if the client sends a message larger than `BUFFER_SIZE` (1024 bytes)?

Answer: Because we specified `BUFFER_SIZE - 1` in `recvfrom()` , the server will only read the first 1023 bytes of the message. The remaining bytes of that specific datagram will be discarded by the operating system.